

IoT Security: An Offensive Case Study

Refik Can OZTAS^{1[n23131610]}

¹ Hacettepe University, Beytepe, Ankara, Turkey

IoT devices are starting to be used more day by day. Although these devices are very useful even with their limited resources, their security is a great attack vector. This paper covers and investigates several attacks on real-life created IoT networks via hardwares to conduct those attacks. Also, as the created network topology, future work directions are also included in this paper.

Keywords: IoT Security, Wireless Security, ZigBee Security, Wi-Fi Security.

1 Introduction

The need to secure the Internet of Things (IoT) is more important than ever as the globe grows more interconnected due to the vast network of smart gadgets. IoT technologies are expanding quickly, which presents a number of opportunities but also raises a serious concern: these linked ecosystems are vulnerable to malevolent threats. This case study explores the obstacles, tactics, and innovations in protecting the confidentiality and integrity of connected devices, delving into the dynamic field of IoT security.

In the age of digital transformation, where IoT interlaces homes, industries, healthcare, and transportation seamlessly, the imperative for steadfast security measures cannot be overstated. My research seeks to untangle the intricacies of IoT security, confronting the distinctive challenges stemming from the diversity of devices, protocols, and communication channels inherent in the IoT terrain.

Notable cases, such as the Mirai botnet attack and the Dyn DDoS attack in 2016, demonstrated the potential havoc that can be wreaked when insecure IoT devices are exploited to orchestrate large-scale disruptions. These incidents exposed critical security flaws in widely used IoT devices, illustrating the urgent need for robust cybersecurity measures.

Although IoT security covers many security areas like Network Security, Hardware Security and many more, and there are other case studies that cover [15] every area, this case study mostly focuses on the Wireless Security and Network Security side of IoT devices.

In order to investigate, research and recreate attack scenarios, firstly, a literature review was conducted. Then, a real basic IoT network was created. After this topology was created, real IoT devices for packet manipulating, sniffing and replaying were set up. Then, various attacks were implemented.

2 Methodology

2.1 Security of IoT devices

A physical object or device that has sensors, software, and connection built in to enable it to communicate and share data with other devices and systems via the internet is referred to as an Internet of Things (IoT) device. These gadgets are a component of a larger ecosystem called the Internet of Things, in which devices may gather, send, and receive data for a variety of uses thanks to their connections and interactions with one another.

IoT devices contain various components. For an attacker, every component of this device can be an attack surface. Basic logical and physical components of an IoT device are:

- **Network**

Since the devices are connected with each other and gateways/routers. Their network connection is a security concern.

- **Web (Front & Backend and Web services)**

Most end-user type IoT devices have web panels in local network and cloud networks for usability. These panels can be important attack vectors.

- **Mobile App (Android & iOS applications)**

Most end-user type IoT devices have mobile applications for usability. These applications can be important attack vectors.

- **Wireless Connectivity (Zigbee, WiFi, Bluetooth, etc)**

The emergence of wireless networking technologies, including WiFi, Bluetooth, Zigbee, and others, has greatly expanded the potential of Internet of Things

devices. But increased connectivity also creates a surface for possible attacks that bad actors could take advantage of. It is essential to comprehend and resolve the vulnerabilities linked to these wireless protocols in order to guarantee the security of IoT ecosystems.

- **Firmware Security (OS of IoT Devices)**

As the devices have firmware that runs their specialized hardware, this firmwares confidentiality and security is an asset.

- **Hardware Hacking & Fault Injections & SCA Attacks**

Some hard-coded information such as master-keys is implemented on hardware level in some cases. This type of information is a target for attackers.

- **Storage Medium**

File system, flash memory and storage medium's security is a concern because they may contain sensitive information.

- **I/O Ports**

Since IoT devices are relatively-old hardwares, they can be vulnerable for binary-exploitation attacks.

This paper focuses on the wireless security of devices.

2.2 IoT network architecture

In order to generate a case study, an IoT network with IoT devices was built. As seen in Figure 1, in wireless communication, IoT devices communicate with ZigBee Gateway via ZigBee protocol. ZigBee Gateway connects to Router via Wi-Fi in order to communicate with Mobile Application and Internet. ZigBee Gateway and IoT devices are relatively-manageable from mobile applications. Mobile application creates a traffic with-in ZigBee gateway in order to receive and send information to IoT endpoints. Although most traffic comes to ZigBee Gateway from Mobile application through Server/Internet, some of the traffic with-in mobile application and

ZigBee gateway are local, therefore, in order to use some of the features, mobile application and ZigBee Gateway must be on the same network.

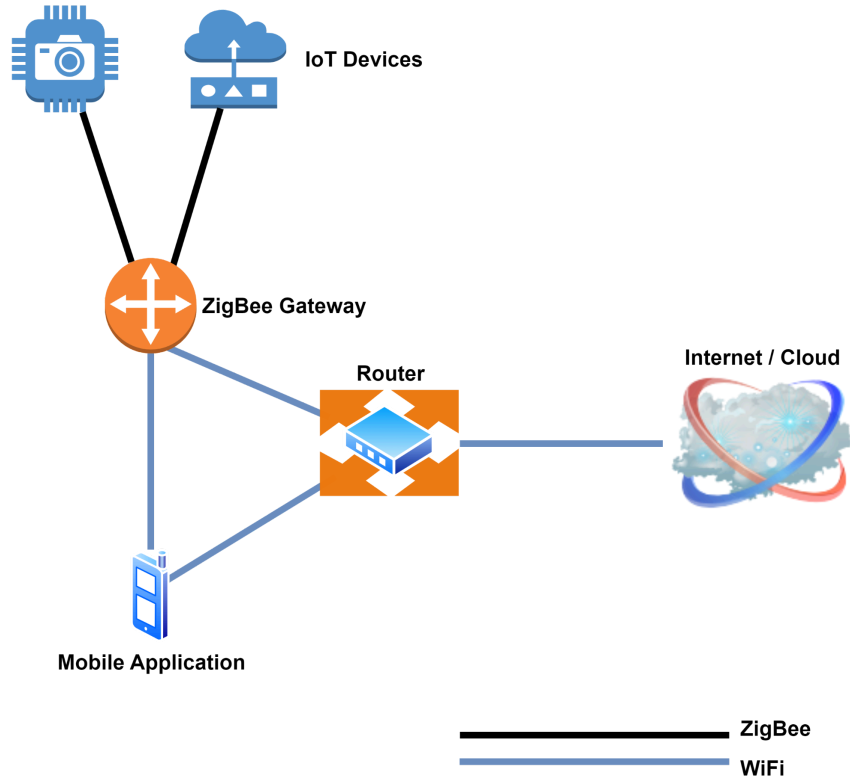


Fig 1: IoT Network Topology

2.2 IoT devices

I selected Imou ZigBee Gateway as Gateway, ZTM1 temperature sensor and Zp1 motion detector as proof-of-concept IoT endpoint devices.



Fig 2: from left to right - Temperature sensor, ZigBee gateway, Motion sensor

2.3 Hardwares For Attacks

In order to generate attacks to wireless networks, additional hardware is required. There is specialized hardware to generate these attacks, or SDR can be used. Although using SDR or different hardware may be debatable based on various metrics, such as performance, implementation difference [1] and some attacks may not be producible on SDR or hardware depending on the attack, I mostly used CC2531 USB-Dongle to produce attacks. Texas Instruments designed the CC2531, a popular wireless system-on-chip (SoC). It is commonly utilized in the development of Zigbee-based applications. Since it is a system-on-chip it requires firmware in order to perform attacks. Details about uploading firmware to the chip are on the appendix.

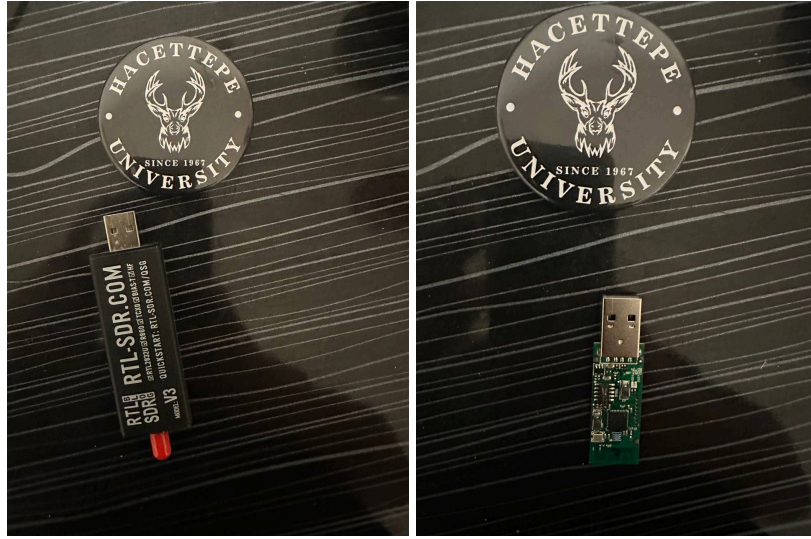


Fig 3: from left to right - RTL-SDR, CC2531 USB-Dongle

3 Implementation

3.1 ZigBee Channel Fuzzing/Scanning

ZigBee uses 2.4GHz channels. Based on the RFC, it starts from 2405MHz (0x0B channel) to 2480MHz (0x1A channel). Although packets can be sent via different channels with-in the same network, most cases use one frequency channel. Listening every channel with time-slots, called Channel Fuzzing/Scanning, is a reconnaissance step of ZigBee attacks.

I listened to all channels with 1-second time slots. In my case channel 0x0E, 2420MHz was used by ZigBee gateway and sensors.

Scan Channels

Scan each channel for: seconds.

☒ Scan All Channels ☐ With Beacon

☒ Zigbee 2.4 GHz

Channel	Frequency	Modulation	Progress
11 (0x0B)	2405 MHz	O-QPSK_250	0%
12 (0x0C)	2410 MHz	O-QPSK_250	0%
13 (0x0D)	2415 MHz	O-QPSK_250	0%
14 (0x0E)	2420 MHz	O-QPSK_250	0%
15 (0x0F)	2425 MHz	O-QPSK_250	0%
16 (0x10)	2430 MHz	O-QPSK_250	0%
17 (0x11)	2435 MHz	O-QPSK_250	0%
18 (0x12)	2440 MHz	O-QPSK_250	0%
19 (0x13)	2445 MHz	O-QPSK_250	0%
20 (0x14)	2450 MHz	O-QPSK_250	0%
21 (0x15)	2455 MHz	O-QPSK_250	0%
22 (0x16)	2460 MHz	O-QPSK_250	0%
23 (0x17)	2465 MHz	O-QPSK_250	0%
24 (0x18)	2470 MHz	O-QPSK_250	0%
25 (0x19)	2475 MHz	O-QPSK_250	0%
26 (0x1A)	2480 MHz	O-QPSK_250	0%

Start

Fig 4: ZigBee Channel Scanning via Ubiquiti Protocol Analyzer

3.2 Network Sniffing

ZigBee protocol sends packets encrypted, however, this encryption can be decrypted easily. [11]. When hardware sets properly, it can act as sender and receiver in ZigBee protocol. [Appendix A]. When traffic is captured, I see encrypted traffic. ZigBee uses two keys because of its protocol. Trust Center Link Key and Network Key. Trust Center key is a generic key in ZigBee protocol. Although some specific vendors like Hue [12], use different trust center keys, it was generic in my case. Detailed instructions are shown in Appendix B [Appendix B].

Network is a secret component, however when a new device is joined to the network, the ZigBee Gateway must send this key to the device in order to communicate. As an attacker, I can sniff this key.

When a device wants to join the network, it sends a broadcast request. When gateway captures these requests, it responds with Permit Join Request, which includes a network key. With the capture key, the attacker can read frames.

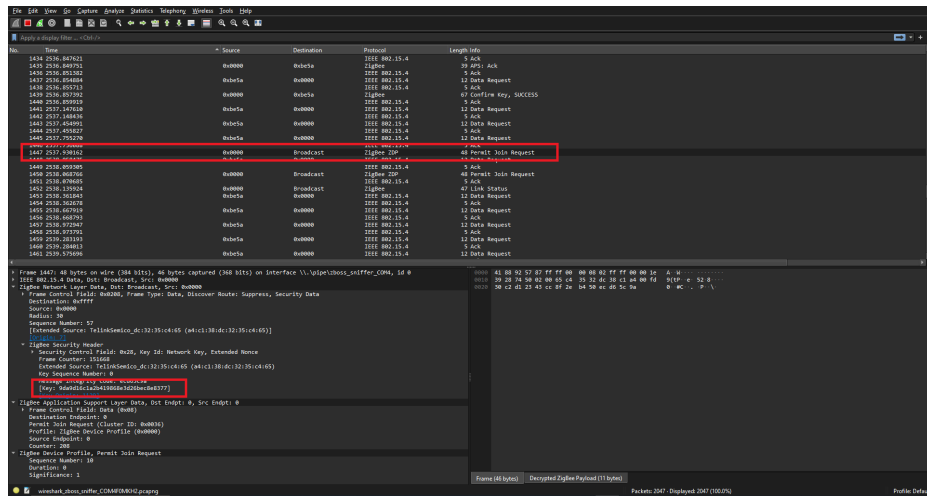


Fig 5: Secret key of the case study in Wireshark

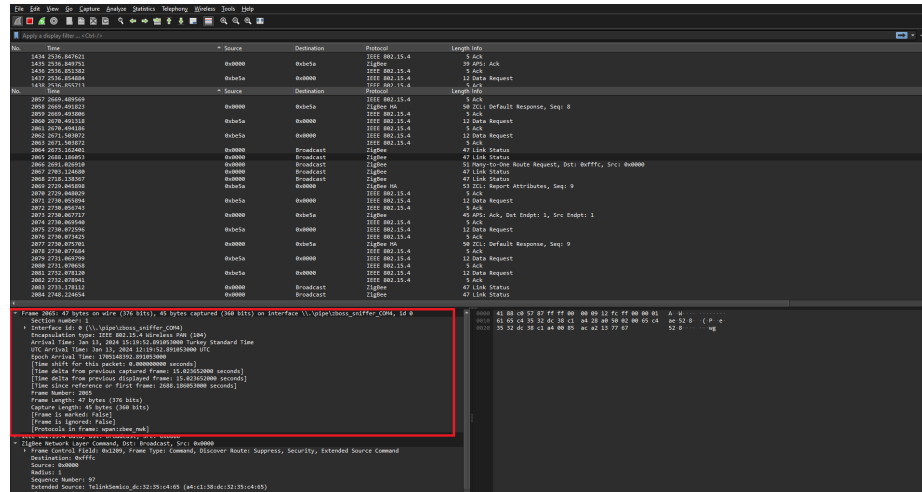


Fig 6: Frame of packets

Traffic View

Select Filter

ID	Ln.	Timestamp	Time DeL.	Ch.	Stack	La..	Packet Information	MAR Src.	MACD...	MAC Se...
1	47	2024/01/13 18:000000	14	ZigB.	NWK		*** Unable to decrypt..	0x0000	0xFFFD	127
	53	2024/01/13 18:2516993	14	ZigB.	NWK		*** Unable to decrypt..	0xBE5A	0x0000	187
2	5	2024/01/13 18:002116	14	ZigB.	MAC		Acknowledgement			187
	53	2024/01/13 18:001675	14	ZigB.	NWK		*** Unable to decrypt..	0xBE5A	0x0000	188
3	5	2024/01/13 18:002212	14	ZigB.	MAC		Acknowledgement			188
	12	2024/01/13 18:092104	14	ZigB.	MAC		Data Request	0xBE5A	0x0000	189
4	5	2024/01/13 18:000828	14	ZigB.	MAC		Acknowledgement			189
	45	2024/01/13 18:001259	14	ZigB.	NWK		*** Unable to decrypt..	0x0000	0xBE5A	128
5	5	2024/01/13 18:001823	14	ZigB.	MAC		Acknowledgement			128
	12	2024/01/13 18:003328	14	ZigB.	MAC		Data Request	0xBE5A	0x0000	190
6	5	2024/01/13 18:000846	14	ZigB.	MAC		Acknowledgement			190
	45	2024/01/13 18:001798	14	ZigB.	NWK		*** Unable to decrypt..	0x0000	0xBE5A	129
7	5	2024/01/13 18:001823	14	ZigB.	MAC		Acknowledgement			129
	45	2024/01/13 18:003570	14	ZigB.	NWK		*** Unable to decrypt..	0x0000	0xBE5A	129
8	12	2024/01/13 18:003639	14	ZigB.	MAC		Data Request	0xBE5A	0x0000	191
	5	2024/01/13 18:000810	14	ZigB.	MAC		Acknowledgement			191
9	45	2024/01/13 18:001277	14	ZigB.	NWK		*** Unable to decrypt..	0x0000	0xBE5A	129
	5	2024/01/13 18:001823	14	ZigB.	MAC		Acknowledgement			129
10	50	2024/01/13 18:001251	14	ZigB.	NWK		*** Unable to decrypt..	0x0000	0xBE5A	130
	12	2024/01/13 18:002197	14	ZigB.	MAC		Data Request	0xBE5A	0x0000	192
11	5	2024/01/13 18:000810	14	ZigB.	MAC		Acknowledgement			192
	50	2024/01/13 18:004561	14	ZigB.	NWK		*** Unable to decrypt..	0x0000	0xBE5A	130
12	5	2024/01/13 18:001983	14	ZigB.	MAC		Acknowledgement			130
	50	2024/01/13 18:001202	14	ZigB.	NWK		*** Unable to decrypt..	0x0000	0xBE5A	131
13	12	2024/01/13 18:004905	14	ZigB.	MAC		Data Request	0xBE5A	0x0000	193
	5	2024/01/13 18:000807	14	ZigB.	MAC		Acknowledgement			193
14	50	2024/01/13 18:001699	14	ZigB.	NWK		*** Unable to decrypt..	0x0000	0xBE5A	131
	50	2024/01/13 18:005194	14	ZigB.	NWK		*** Unable to decrypt..	0x0000	0xBE5A	131
15	50	2024/01/13 18:005013	14	ZigB.	NWK		*** Unable to decrypt..	0x0000	0xBE5A	131
	12	2024/01/13 18:0952497	14	ZigB.	MAC		Data Request	0xBE5A	0x0000	194
16	5	2024/01/13 18:000819	14	ZigB.	MAC		Acknowledgement			194
	12	2024/01/13 18:1009012	14	ZigB.	MAC		Data Request	0xBE5A	0x0000	195
17	5	2024/01/13 18:000820	14	ZigB.	MAC		Acknowledgement			195
	53	2024/01/13 18:017687	14	ZigB.	NWK		*** Unable to decrypt..	0xBE5A	0x0000	196
18	5	2024/01/13 18:003129	14	ZigB.	MAC		Acknowledgement			196
	12	2024/01/13 18:1006473	14	ZigB.	MAC		Data Request	0xBE5A	0x0000	197
19	5	2024/01/13 18:000823	14	ZigB.	MAC		Acknowledgement			197
	45	2024/01/13 18:002073	14	ZigB.	NWK		*** Unable to decrypt..	0x0000	0xBE5A	132
20	5	2024/01/13 18:001823	14	ZigB.	MAC		Acknowledgement			132
	12	2024/01/13 18:002662	14	ZigB.	MAC		Data Request	0xBE5A	0x0000	198
21	5	2024/01/13 18:000832	14	ZigB.	MAC		Acknowledgement			198
	50	2024/01/13 18:002023	14	ZigB.	NWK		*** Unable to decrypt..	0x0000	0xBE5A	133
22	5	2024/01/13 18:000808	14	ZigB.	MAC		Acknowledgement			198
	50	2024/01/13 18:002023	14	ZigB.	NWK		*** Unable to decrypt..	0x0000	0xBE5A	133

Traffic View

Results View

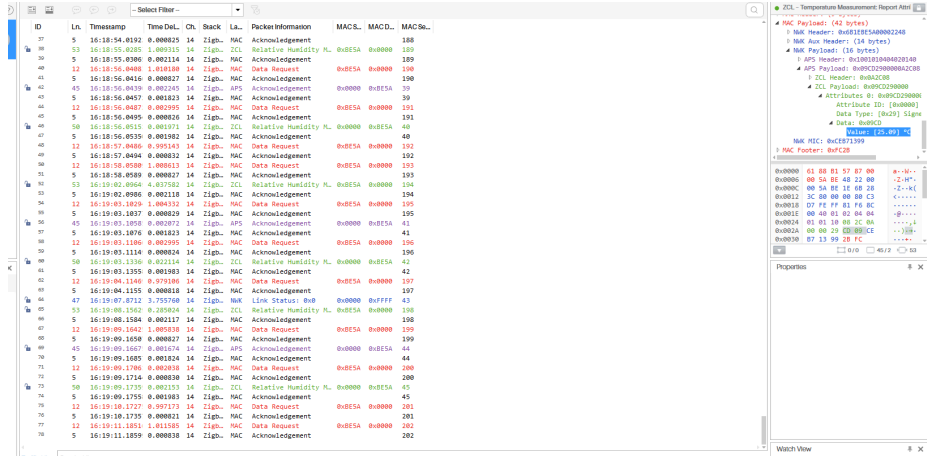


Fig 7: Encrypted packets and frame of packets in Ubique Packet Analyzer

3.2 Packet Injection/Manipulation

Since a packet can be unencrypted and readable, it can be manipulated also. For instance, in our network, sensors send temperature data to ZigBee Gateway. Although jamming a specific packet and forging a manipulated one is not easily-implementable, sending newly crafted packets can manipulate Gateway, ultimately the whole architecture. For instance, if there is a rule that enables a cooling-system in a specific temperature, it can be easily triggered.

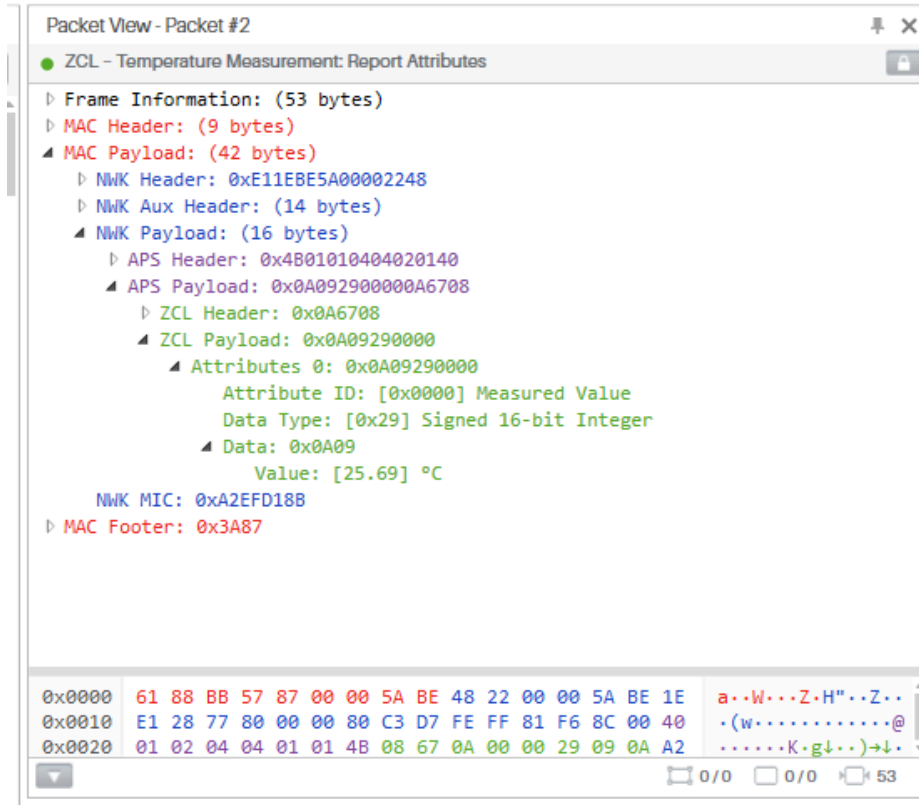


Fig 8: Example ZigBee Packet from Sensor to Gateway

3.4 Wi-Fi Protocol of ZigBee Gateway

WiFi deauthentication (deauth) attack is a type of cyber attack that targets wireless networks. The goal of this attack is to disconnect devices from a WiFi network by sending deauthentication frames to the targeted devices. These frames essentially tell the devices that they are no longer authenticated to the network, forcing them to disconnect and prompting them to reauthenticate.

Our GateWay uses Wi-Fi to communicate with the internet, and it uses a 2.4Ghz channel. Therefore, when we disconnect the 2.4Ghz channel, our GateWay will not be able to communicate with the internet.

This attack causes Denial of Service with connection to the cloud. Our alarm rings since they are connected to the gateway via ZigBee, however, we can not get alarm notification from mobile applications since the IoT networks cloud communication was lost.

Detailed steps can be found on Appendix C [Appendix C]

3.5 Other Component Findings

Other findings are listed in Appendix D [Appendix D].

4 Related Works

In general, wireless attacks may be classified into three basic section:

- Packet sniffing
- Packet injection
- Packet manipulation

Sniffing can be considered passive reconnaissance in the offensive-security methodology. Only confidentiality of the data is an attack surface. An attacker can sniff packets without generating any logs.

Injection and manipulation can be considered more complex attacks. It aims at Confidentiality, Integrity and Availability of data.

Although not every ZigBee attack is directly related to ZigBee protocol, it can be caused by another layer protocols, based on the paper by M. Gupta, S. Singh [3], general ZigBee Weaknesses and Suggestions [2], Routing Layer Intrusions [4], ZigBee Vulnerabilities [5], Zigbee Topology [6], Zigbee Concepts [7], General Overview [8], Key Management [9], Zigbee and IEEE 802.15.4 [10], Zigbee Attack Classification [3], are the mostly studied areas in the domain of ZigBee.

5 Future Work Direction

My case study required comparison and merging of various attacks on the Wireless Security side of IoT devices. Based on this general study, future direction on this research might be:

- Since there is a PoC network and architecture for ZigBee, a dataset can be created from real ZigBee traffic.
- Based on this traffic, implemented attacks can be generated in the traffic. And, study for solving anomaly detection problems, classification of attack problems can be conducted via Machine Learning / Artificial Intelligence / Neural Network based approaches.
- Although this paper covers mostly wireless attacks of IoT devices, other security components might be studied as a case study.
- Based on the traffic, IPS/IDS systems can be formed.
- With a proper IDS/IPS system, if hardware of devices is also studied, there can be Endpoint Based Security Solutions for such systems.
- In addition, as a deep-dive security approach, vulnerabilities on ZigBee protocol, or physically attack vectors (such as jamming with SDR) could be studied and analyzed.

6 Conclusion

My research is a basic security case study on IoT devices. This research starts with definitions and IoT devices security of components. Although this paper mostly covers the wireless security of IoT devices, other components security is also mentioned. In order to conduct a case study, I created an IoT network with various devices, made the configuration and set-up the hardware devices in order to conduct ZigBee attacks.

As mentioned in future directions, this case study may be extended to generate dataset, characterize dataset and furthermore. My generated ZigBee data (not all) is referenced in Appendix E [Appendix E].

Although ZigBee is a cheap technology for low-resource devices, its security in IoT seems problematic. Hong Kong CERT team published a security guideline [13] to prevent attacks just like in this case study, however, since the area like IoT is very vendor dependent, forming and conducting security principles may not be possible in near future.

References

1. Dalrymple, Scott D., "Comparison of ZigBee Replay Attacks Using a Universal Software Radio Peripheral and USB Radio" (2014). Theses and Dissertations. 595. <https://scholar.afit.edu/etd/595>
2. Alireza Zohourian, Sajjad Dadkhah, Euclides Carlos Pinto Neto, Hassan Mahdikhani, Priscilla Kyei Danso, Heather Molyneaux, Ali A. Ghorbani, IoT Zigbee device security: A comprehensive review, *Internet of Things*, Volume 22,2023,100791, ISSN 2542-6605, <https://doi.org/10.1016/j.iot.2023.100791>.(<https://www.sciencedirect.com/science/article/pii/S2542660523001142>)
3. M. Gupta, S. Singh, A survey on the zigbee protocol, it's security in internet of things (iot) and comparison of zigbee with bluetooth and wi-fi, in: *Applications of Artificial Intelligence in Engineering*, Springer, 2021, pp. 473–482.
4. N. Sidhu, M. Sachdeva, A comprehensive study of routing layer intrusions in zigbee based wireless sensor networks, *Int. J. Adv. Sci. Technol.* 29 (3) (2020) 514–524.
5. S. Khanji, F. Iqbal, P. Hung, ZigBee security vulnerabilities: Exploration and evaluating, in: *2019 10th International Conference on Information and Communication Systems, ICICS, IEEE*, 2019, pp. 52–57.
6. T. Kumar, P. Mane, ZigBee topology: A survey, in: *2016 International Conference on Control, Instrumentation, Communication and Computational Technologies, ICCICCT, IEEE*, 2016, pp. 164–166.
7. J.M. Varghese, K. Nibi, V.T. Varghese, S. Rao, A survey of the state of the art in ZigBee, *Int. J. Cybern. Inf.* 4 (2) (2015) 145–155.
8. O.G. Aju, A survey of zigbee wireless sensor network technology: Topology, applications and challenges, *Int. J. Comput. Appl.* 130 (9) (2015) 47–55.
9. Z.A. Davani, A.A. Manaf, A survey on key management of ZigBee network, in: *Proceedings of the International Conference on E-Technologies and Business on the Web (EBW'2013)*, Bangkok, Thailand, Citeseer, 2013, pp. 7–9.
10. P. Baronti, P. Pillai, V.W. Chook, S. Chessa, A. Gotta, Y.F. Hu, Wireless sensor networks: A survey on the state of the art and the 802.15. 4 and ZigBee standards, *Comput. Commun.* 30 (7) (2007) 1655–1695.
11. Xueqi Fan, Fransisca Susan, William Long, Shangyan Li, Security Analysis of Zigbee, MIT
12. <https://peeveeone.com/2016/11/breakout-breakthrough/>
13. <https://www.hkcert.org/f/guideline/264461/3a1c8eed-012c-4b59-9d9e-971001d66c77-DL-FE-14602.pdf>
<https://www.hkcert.org/security-guideline/implementing-iot-security-best-practice>
- 14.
15. Philipp Morgner, Stephan Mattejat, Zinaida Benenson, All Your Bulbs Are Belong to Us: Investigating the Current State of Security in Connected Lighting Systems, arXiv:1608.03732v2 [cs.CR] 21 Jun 2017
16. Lennart Jansen, Assessing smart home security: a Zigbee case study, University of Twente, https://essay.utwente.nl/89274/1/89274_Jansen_MA_EEMCS.pdf

APPENDIX A

Hardware Setup

I used a CC2531 USB Dongle in order to capture and send packets. Although its hardware is specialized for ZigBee, in order to work with real-life cases it needs firmware. Firmwares of CC2531 is directly related to which packet capture technology you are using. Therefore, in the research process, I tried various packet capture softwares and firmwares.

In order to install firmware, CC Debugger by Texas Instruments are required. In addition, debugger has 10 pins and the CC2531 has 8, therefore a transformer cable is also required.

After that, with Texas Instruments software, firmware can be uploaded to SoC, and related packet sniffers can be used.

In this paper, if Ubiquia protocol analyzer is used in the attack, CC2531 uses ubqia-sn_v0.9.0.hex firmware. If wireshark is used, it uses zboss_sniffer.hex firmware.

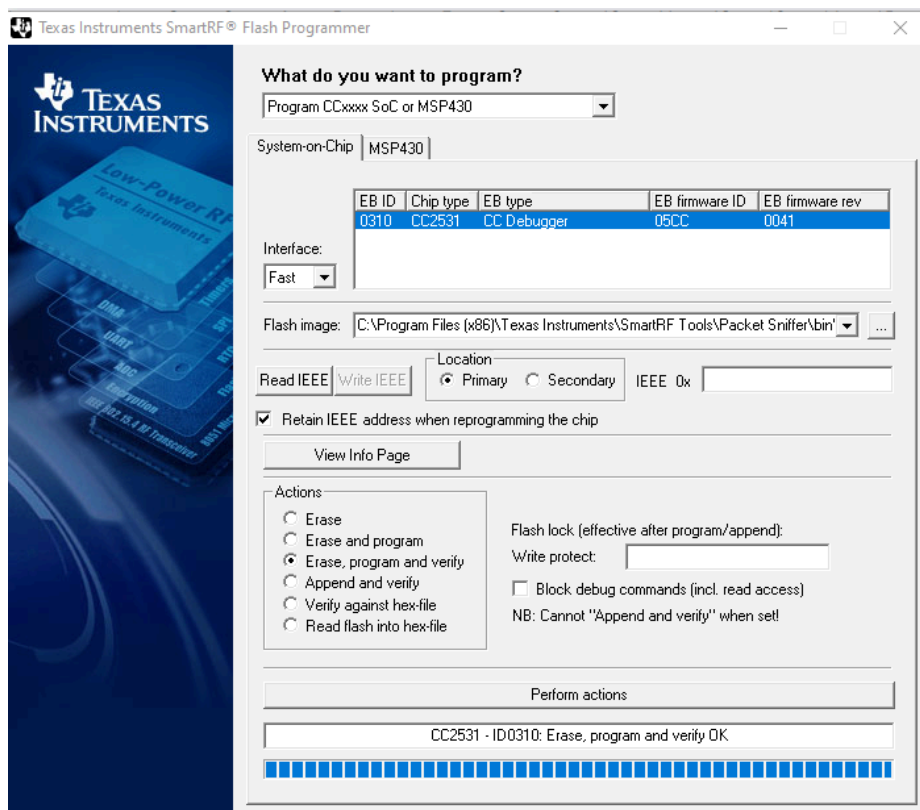


Fig A.1: Firmware Flash Programmer Software by Texas Instruments

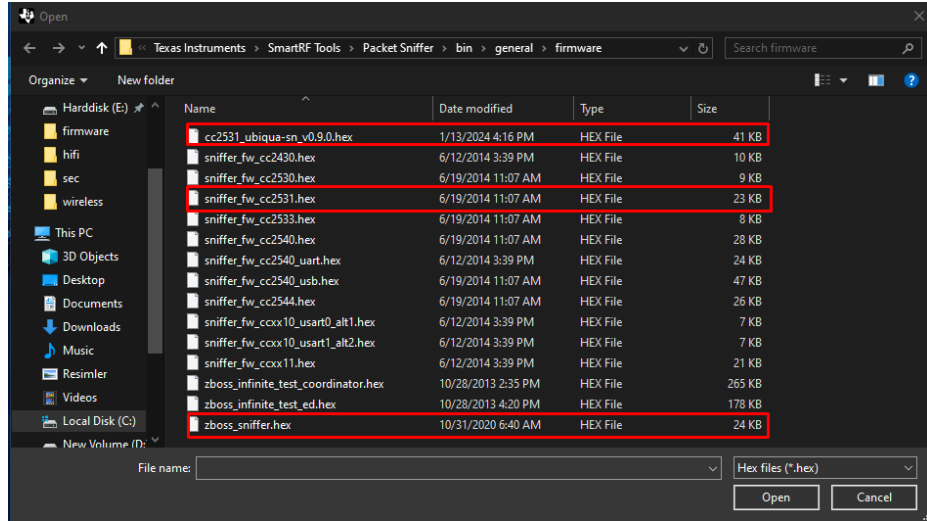


Fig A.2: Firmware list I used in this case by various vendors

APPENDIX B

Packet Sniffing Setup

Although they are very-similar, I used three packet sniffer softwares in this study. Texas Instruments SmartRF packet sniffer, which I do not think works stable, Wireshark and Ubiquia protocol analyzer. Ubiquia protocol analyzer is an enterprise solution which I used in trial mode, is very user-friendly. On the other hand wireshark is the most common packet-inspection tool for not just IoT networks, for IT and OT networks too.

The screenshot displays the Texas Instruments SmartRF Packet Sniffer interface. The main window shows a list of captured packets. The first packet is a Frame control frame (Type: Sync, Protocol: ZigBee, Length: 11). The second packet is a MAC payload frame (Type: Sync, Protocol: ZigBee, Length: 11). The third packet is an Encrypted MAC payload frame (Type: Sync, Protocol: ZigBee, Length: 11). The details pane on the right shows the structure of the selected packet, including fields like Sequence number, Frame counter, and Key identifier.

Fig B.1: Encrypted packets on SmartRF packet sniffer

The screenshot displays the Wireshark interface. The main window shows a list of captured packets. The first packet is a Broadcast frame (Type: Broadcast, Length: 1). The second packet is a ZigBee frame (Type: ZigBee, Length: 1). The third packet is an IEEE 802.15.4 frame (Type: IEEE 802.15.4, Length: 1). The details pane on the right shows the structure of the selected packet, including fields like Frame type, Frame counter, and Key identifier.

Fig B.2: Packets in Wireshark

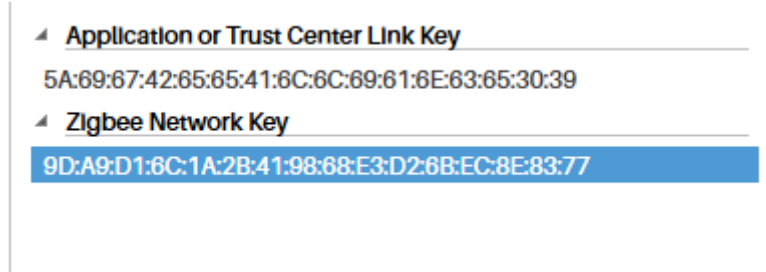


Fig B.3: Network key configuration in Ubiquia Protocol Analyzer

APPENDIX C

```

sudo airodump-ng -bg -
CH 12 ][ Elapsed: 3 mins ][ 2023-12-16 11:58 ][ interface wlan0 down

BSSID          PWR Beacons  #Data  #/s  CH  MB  ENC CIPHER AUTH ESSID
40:ED:00:34:D0:C0 -42    237      36    0  10  130 WPA2 CCMP PSK  TurKTelekom_TFD0BA_2.4GH

BSSID          STATION          PWR   Rate    Lost    Frames  Notes  Probes
40:ED:00:34:D0:C0 34:94:54:F2:CB:70 -52    0 - 6      0      120
40:ED:00:34:D0:C0 EE:D1:90:F1:E5:59 -31    1e-24    591     102
40:ED:00:34:D0:C0 62:36:2A:60:FE:2C -27    1e-1e    11     310

```

Fig C.1: Finding MAC address of the GateWay

```

4 sudo aireplay-ng --deauth 100000 -a 40:ED:00:34:D0:C0 -c 34:94:54:F2:CB:70 wlan0 -p
12:02:27 Sending 64 directed DeAuth (code 7). STMAC: [34:94:54:F2:CB:70] [29 39 ACKs]
12:02:28 Sending 64 directed DeAuth (code 7). STMAC: [34:94:54:F2:CB:70] [ 1 5 ACKs]
12:02:28 Sending 64 directed DeAuth (code 7). STMAC: [34:94:54:F2:CB:70] [ 0 0 ACKs]
12:02:29 Sending 64 directed DeAuth (code 7). STMAC: [34:94:54:F2:CB:70] [ 0 0 ACKs]
12:02:29 Sending 64 directed DeAuth (code 7). STMAC: [34:94:54:F2:CB:70] [ 0 0 ACKs]
12:02:30 Sending 64 directed DeAuth (code 7). STMAC: [34:94:54:F2:CB:70] [ 0 0 ACKs]
12:02:30 Sending 64 directed DeAuth (code 7). STMAC: [34:94:54:F2:CB:70] [ 0 0 ACKs]
12:02:31 Sending 64 directed DeAuth (code 7). STMAC: [34:94:54:F2:CB:70] [ 0 0 ACKs]
12:02:32 Sending 64 directed DeAuth (code 7). STMAC: [34:94:54:F2:CB:70] [ 0 0 ACKs]
12:02:32 Sending 64 directed DeAuth (code 7). STMAC: [34:94:54:F2:CB:70] [ 0 0 ACKs]
12:02:33 Sending 64 directed DeAuth (code 7). STMAC: [34:94:54:F2:CB:70] [ 0 0 ACKs]
12:02:34 Sending 64 directed DeAuth (code 7). STMAC: [34:94:54:F2:CB:70] [ 0 0 ACKs]
12:02:35 Sending 64 directed DeAuth (code 7). STMAC: [34:94:54:F2:CB:70] [ 2 0 ACKs]
12:02:35 Sending 64 directed DeAuth (code 7). STMAC: [34:94:54:F2:CB:70] [ 0 0 ACKs]
12:02:36 Sending 64 directed DeAuth (code 7). STMAC: [34:94:54:F2:CB:70] [ 0 0 ACKs]
12:02:37 Sending 64 directed DeAuth (code 7). STMAC: [34:94:54:F2:CB:70] [ 0 0 ACKs]
12:02:38 Sending 64 directed DeAuth (code 7). STMAC: [34:94:54:F2:CB:70] [43 46 ACKs]
12:02:38 Sending 64 directed DeAuth (code 7). STMAC: [34:94:54:F2:CB:70] [ 2 19 ACKs]
12:02:39 Sending 64 directed DeAuth (code 7). STMAC: [34:94:54:F2:CB:70] [ 6 0 ACKs]
12:02:39 Sending 64 directed DeAuth (code 7). STMAC: [34:94:54:F2:CB:70] [ 0 0 ACKs]
12:02:40 Sending 64 directed DeAuth (code 7). STMAC: [34:94:54:F2:CB:70] [ 0 0 ACKs]
12:02:41 Sending 64 directed DeAuth (code 7). STMAC: [34:94:54:F2:CB:70] [ 0 0 ACKs]
12:02:42 Sending 64 directed DeAuth (code 7). STMAC: [34:94:54:F2:CB:70] [ 0 0 ACKs]
12:02:43 Sending 64 directed DeAuth (code 7). STMAC: [34:94:54:F2:CB:70] [ 0 0 ACKs]

```

Fig C.2: Performing DeAuth Attack



Fig C.3: Gateway turns into red color because of Wi-Fi connection is lost

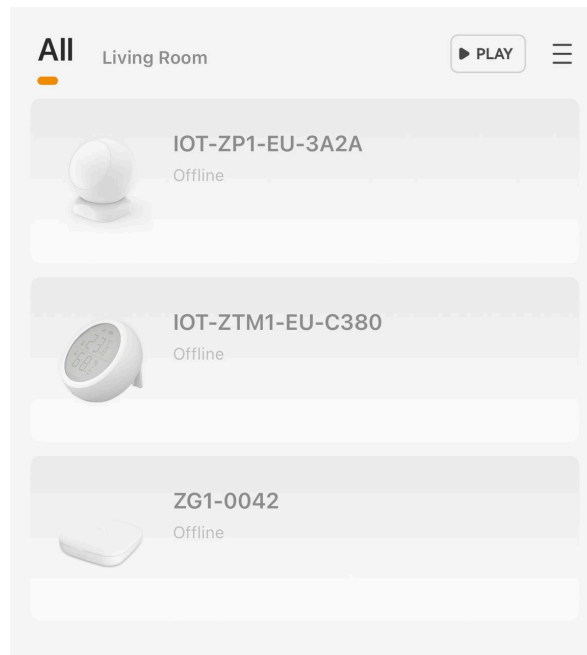


Fig C.4: Mobile Application (Cloud) loses connection because of DeAuth attack

APPENDIX D

Gathering Information via Man-in-the-Middle:

I poisoned the arp-table in the network in order to see the traffic between Gateway, Router(internet) and mobile application.

Therefore, I discovered the cloud IP of the main server:

[illegible]

Fig D.1: Wireshark output of L4 traffic of network

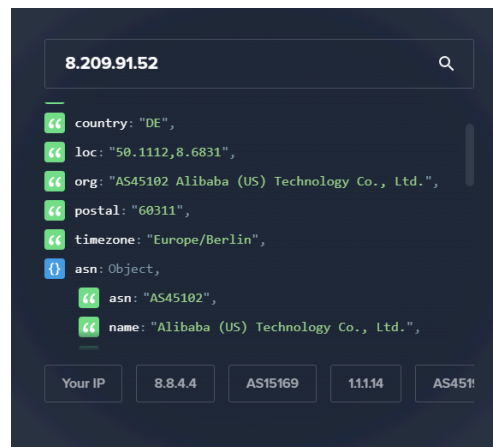
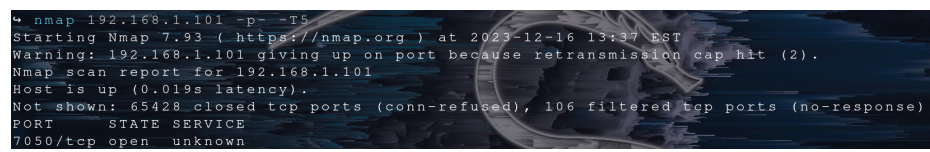


Fig D.2: IPInfo of cloud server

Network port scan:

I conducted a full port scan for the ZigBee gateway, I discovered that port 7050 is open in the device.



```
4 nmap 192.168.1.101 -p- -T5
Starting Nmap 7.93 ( https://nmap.org ) at 2023-12-16 13:37 EST
Warning: 192.168.1.101 giving up on port because retransmission cap hit (2).
Nmap scan report for 192.168.1.101
Host is up (0.019s latency).
Not shown: 65428 closed tcp ports (conn-refused), 106 filtered tcp ports (no-response)
PORT      STATE SERVICE
7050/tcp  open  unknown
```

Fig D.3: nmap output of ZigBee Gateway

APPENDIX E

<https://canoztas.github.io/zigbeedata1.pcapng>

<https://canoztas.github.io/zigbeedata2.cubx>

<https://canoztas.github.io/zigbeedata3.psd>